

Personal Finance & Expense Tracker

A Command-Line Java Application for Income, Expense,
Budget & Recurring-Transaction Management

Project Report

Course: Programming In Java

Submitted by: shivika patidar

Registration No.: 25BAI11010

Date: 18 September 2026

1. Introduction

Managing personal finances sounds simple in theory—spend less than you earn—but in practice, the hard part is visibility. Most people don't overspend because they're careless with money; they overspend because tracking a dozen small transactions a week by hand is tedious enough that they stop doing it within a few weeks. By the time the bank statement shows up at the end of the month, the useful moment to course-correct has already passed.

This project is a command-line Personal Finance & Expense Tracker built in core Java. It's a deliberately lightweight alternative to bloated mobile finance apps: no bank linking, no ads, no account creation—just a fast menu-driven tool that logs transactions, tracks recurring payments, checks spending against budgets in real time, and remembers everything locally between sessions.

The project was built to satisfy two things at once: a genuinely usable personal tool, and a demonstration of the object-oriented and exception-handling concepts covered in the Programming in Java syllabus—custom checked exceptions, a layered package structure, enum-driven modeling, and file-based persistence without any external database.

2. Problem Statement

Manual expense tracking fails because it's tedious and because feedback comes too late (after the month closes, not while a budget is being approached). Existing consumer apps solve this but are heavier than necessary for a single user who just wants a fast logging tool. This project builds a CLI-based tracker that supports transaction logging, category-based monthly budgets with WARNING/CRITICAL alerts, recurring transaction templates, and local flat-file persistence—all using only the Java standard library.

3. Objectives

- **Zero External Dependencies:** Build a fully offline, command-line personal finance tracker using only the Java standard library (no external dependencies, no database server).
- **Full Lifecycle Management:** Support the complete lifecycle of a transaction: creation, validation, categorization, viewing/filtering, and deletion, with balance always kept consistent.
- **Proactive Budget Alerts:** Let the user define monthly, per-category budgets and receive timely WARNING/CRITICAL alerts based on actual spending, rather than discovering an overspend only after the fact.
- **Automated Recurring Entries:** Automate recurring, predictable transactions (rent, subscriptions, salary) so they don't need to be re-entered every period.
- **Exportable Reporting:** Produce human-readable summary reports and exportable CSV/text output for record-keeping outside the application.
- **Resilient Local Persistence:** Persist all application state locally between runs, tolerating minor data corruption gracefully instead of failing outright.
- **Clean Software Architecture:** Demonstrate sound object-oriented design and exception-handling practice: a layered package structure (model / service / util / exception), enum-driven domain modeling, and purpose-built checked exceptions in place of generic ones.

4. Functional Requirements

#	Requirement	Implemented In
FR1	Record income/expense transactions with amount, category, date, description	Transaction ,Account.addTransaction()
FR2	Validate transactions (positive amount, non-null category/date, sanity cap on size) before accepting them	Account.validate() , InvalidTransactionException
FR3	View all transactions, or filter by category	Account.getAllTransactions() , getTransactionsByCategory()
FR4	Delete a transaction by ID and correctly reverse its effect on balance	Account.removeTransaction()
FR5	Set/update a monthly spending limit per category	Budget ,MainApp.setOrUpdateBudget()
FR6	Evaluate spending against budgets and raise WARNING (≥80%) / CRITICAL (≥100%) alerts	AlertEngine.evaluate() ,checkAllBudgets()
FR7	Define recurring transaction templates (daily/weekly/monthly/yearly)	RecurringTransaction
FR8	Materialize due recurring transactions into real transactions on demand	RecurringTransaction.materialize() , MainApp.processDueRecurringTransactions()
FR9	Generate a monthly summary report (totals, category breakdown, budget status)	ReportGenerator.generateSummaryReport()
FR10	Export transactions and category summaries to CSV, and	ReportGenerator.exportTransactionsCsv() exportCategorySummaryCsv() ,

	reports to files	text	exportTextReport()
FR11	Persist all and reload automatically on next run	data locally it	DataStorageUtil

5. Non-Functional Requirements

1. Non-Functional Requirements

- **Reliability:** Corrupted rows in a data file are skipped individually on load (with a warning printed), rather than failing the entire load or crashing the application. IDs loaded from disk are also re-synced against the in-memory ID counters (Transaction.ensureNextIdAtLeast) so restarts never produce duplicate transaction IDs.
- **Error-Handling Strategy:** Invalid domain states are represented as dedicated checked exceptions (InvalidTransactionException, BudgetExceededException) rather than generic RuntimeExceptions, forcing every caller to explicitly decide how to handle a bad transaction or a blown budget instead of letting it propagate silently.
- **Usability:** The CLI menu is numbered and redisplayed after every action with the current balance visible, so the user always has context without needing to remember prior output. Defaults (e.g., "press Enter for today's date") reduce typing for the common case.
- **Maintainability:** The codebase is split into four packages by responsibility (model, service, util, exception), so, for example, changing how reports are formatted never requires touching persistence code, and adding a new alert rule never requires touching the Account class.
- **Performance / Resource Efficiency:** No external dependencies, in-memory processing with $O(n)$ scans over a List<Transaction> sized for a realistic personal transaction volume (hundreds to low thousands of records), well within what's instant on a laptop.
- **Portability:** Pure JDK (java.time, java.nio.file), no build tool or external library required; compiles and runs anywhere a JDK 17+ is installed.

6. System Architecture

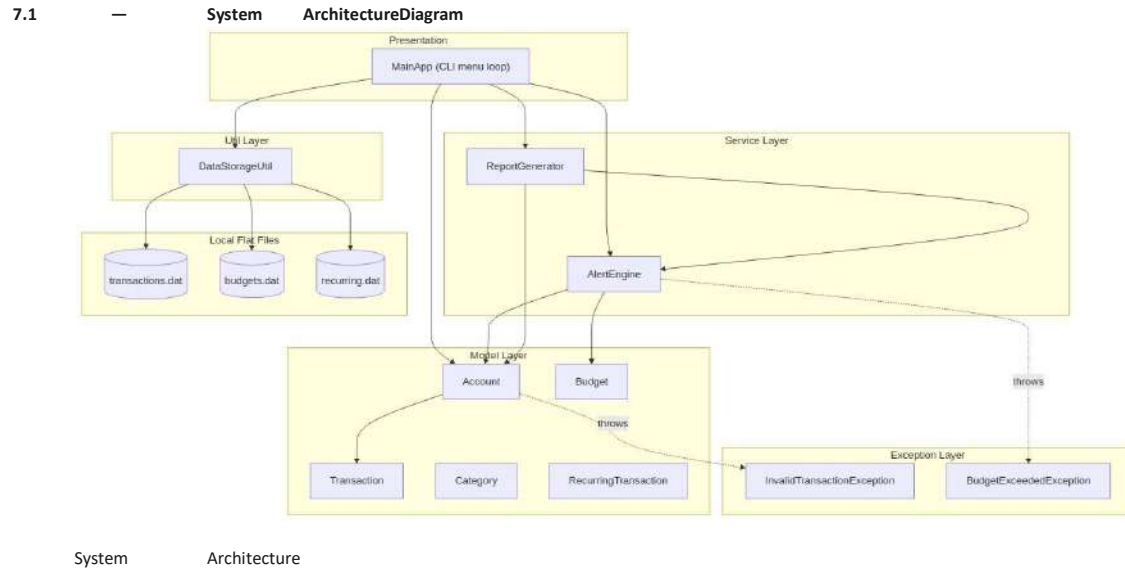
The application follows a simple layered architecture:

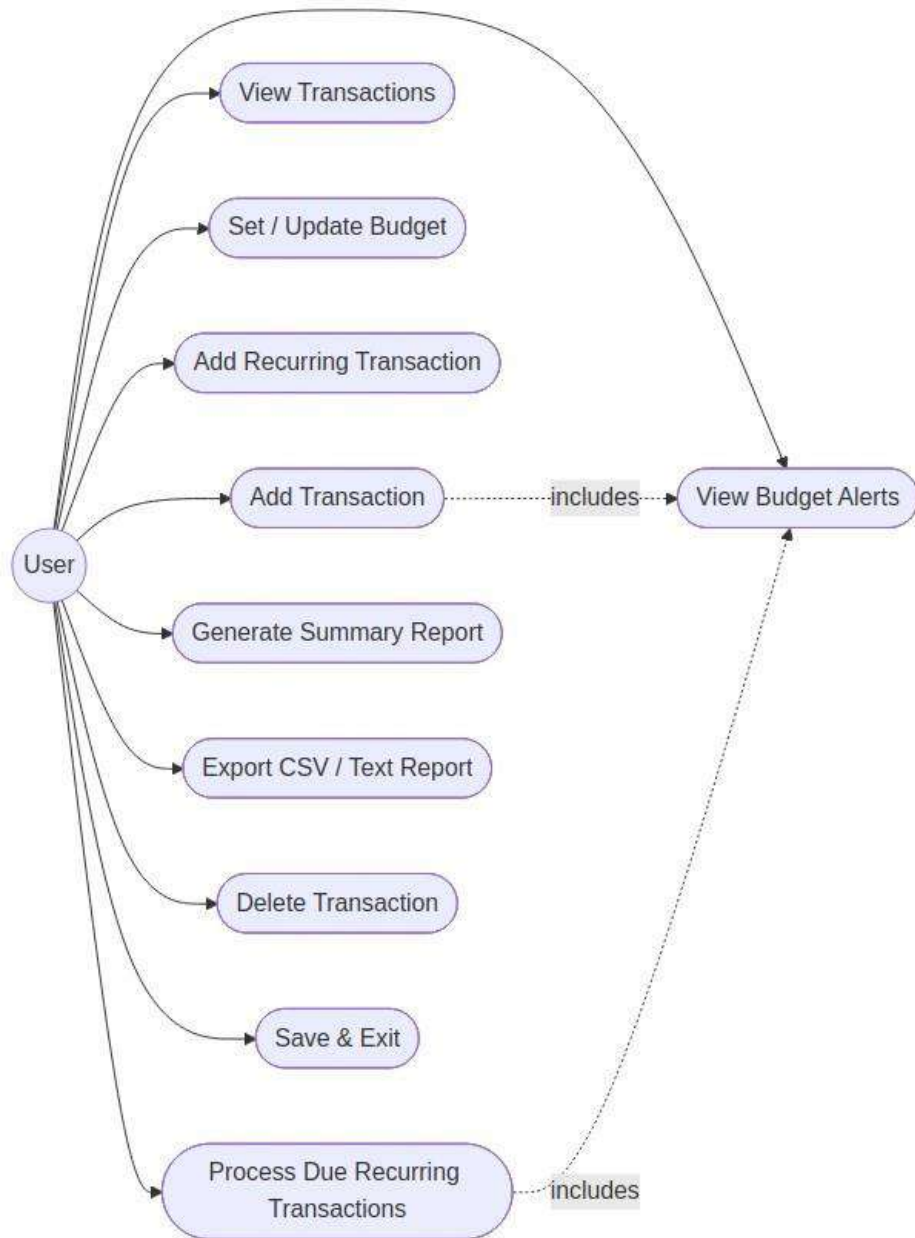
- **Presentation Layer (main):** MainApp owns the menu loop, reads user input via Scanner, and delegates every actual operation to the service/model layer. It contains no business logic of its own.
- **Service Layer (service):** AlertEngine contains the budget-evaluation rules; ReportGenerator builds human-readable reports and CSV exports from model data.
- **Model Layer (model):** Account, Transaction, Budget, RecurringTransaction, and Category hold the actual domain state and encapsulate their own validation/serialization logic (toStorageRow / fromStorageRow).
- **Util Layer (util):** DataStorageUtil is the only class that touches the filesystem for application data, keeping I/O concerns out of the model and service layers.

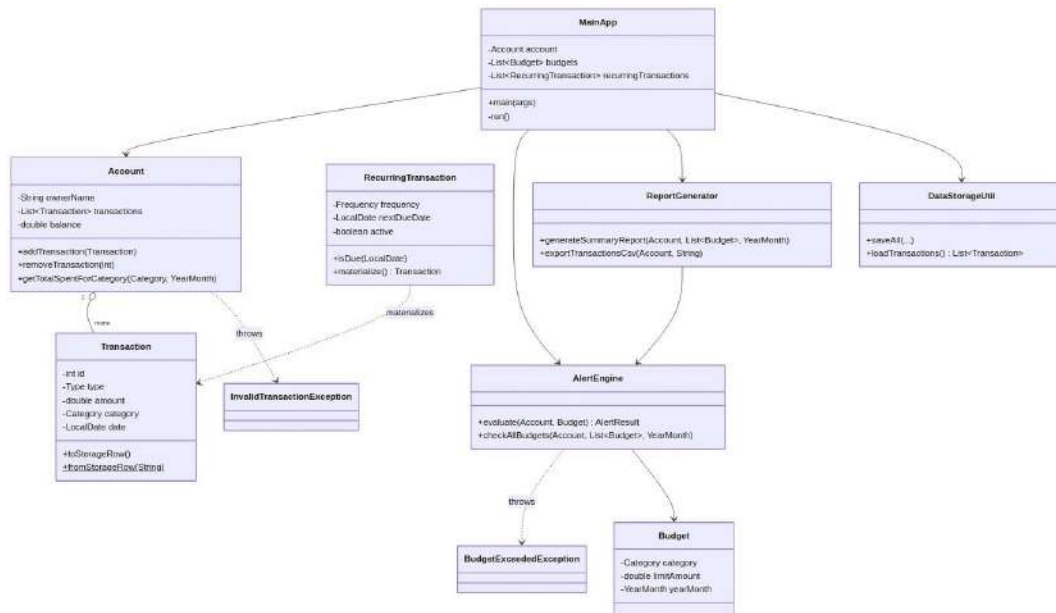
- **Exception Layer (exception):** Two purpose-built checked exceptions that model domain-specific failure states.

See docs/diagrams.md for the rendered system architecture diagram.

7. Design Diagrams



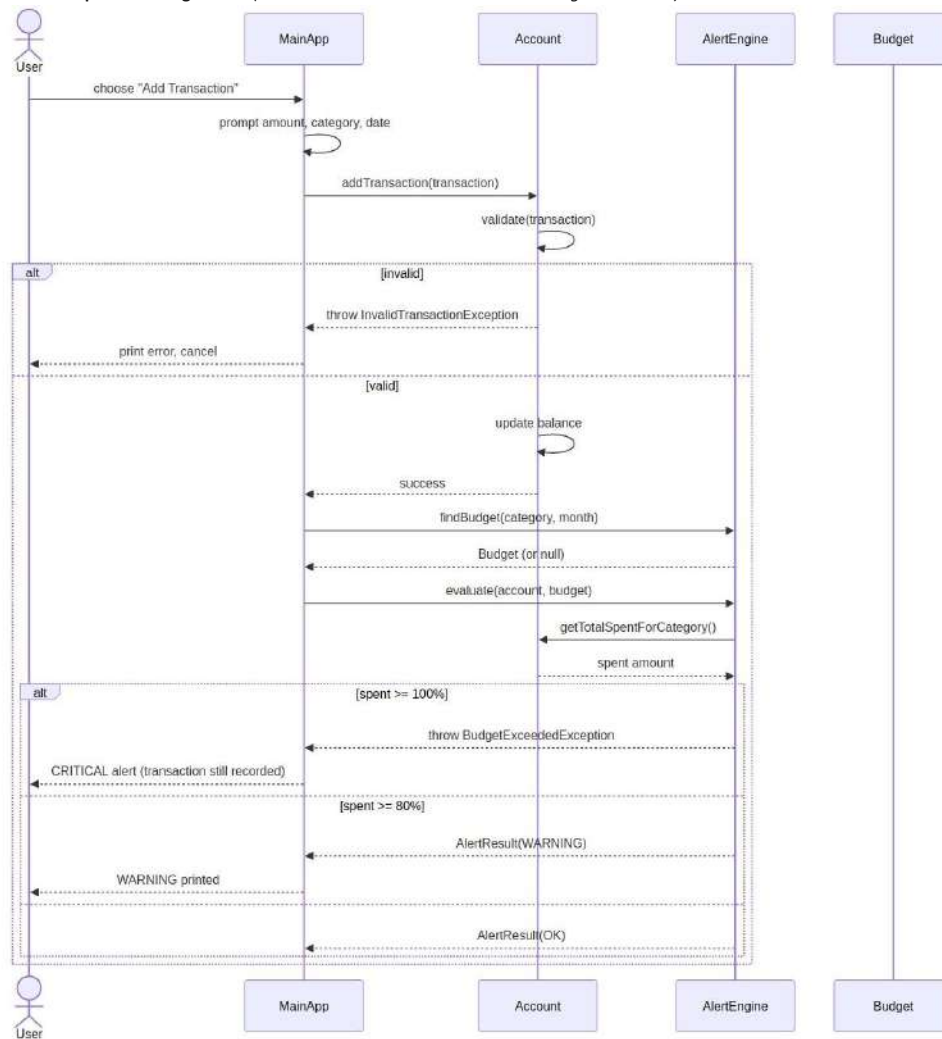




Class Diagram

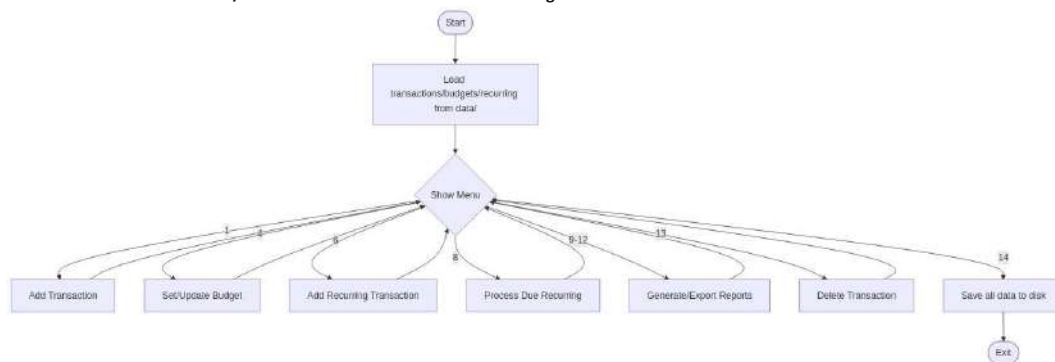
7.4

— Sequence Diagram (Add Transaction → Budget Alert)

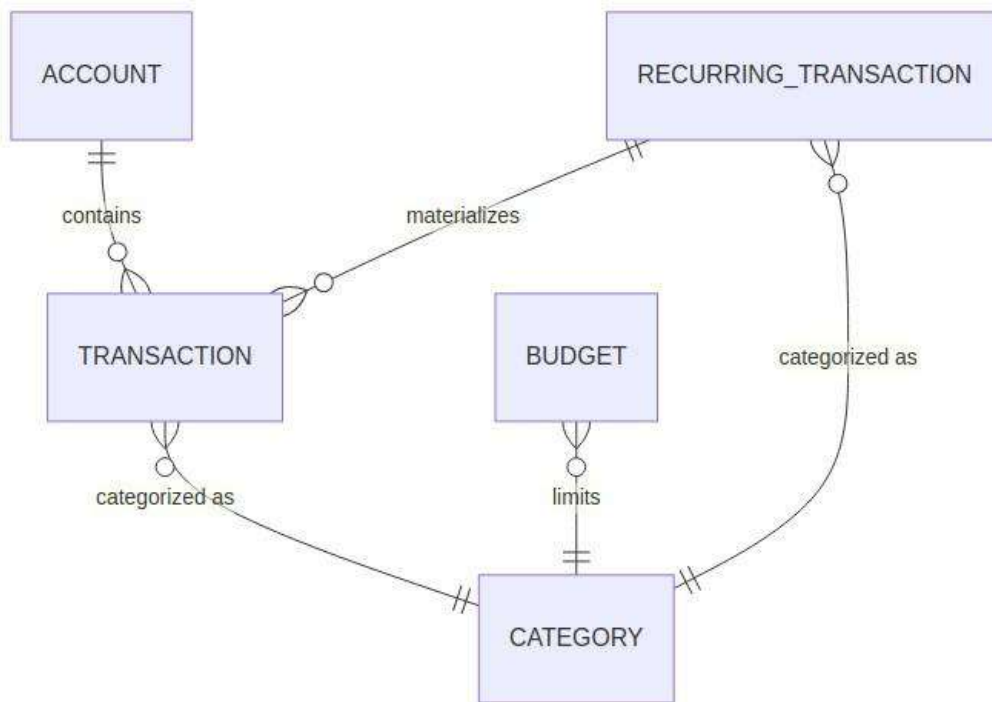


7.5

Sequence Diagram Workflow / Process Flow Diagram



Workflow Diagram



8. Design Decisions & Rationale

Here is your text cleaned up, re-formatted, and structured for clear readability while maintaining all your original technical details.

1. Design Decisions & Rationale

- Checked exceptions over unchecked:** `InvalidTransactionException` and `BudgetExceededException` are both checked, which forces every call site in `MainApp` to explicitly handle them. This avoids letting bad amounts silently propagate as unchecked exceptions that could potentially crash the CLI loop. This was a deliberate trade-off: accepting extra try/catch boilerplate in exchange for guaranteed handling at the boundary where user input enters the system.
- Flat-file, pipe-delimited storage instead of a database:** The scope of this project (single user, no concurrent access) does not require a full database. A pipe-delimited format was chosen over JSON specifically so the project maintains zero external dependencies and compiles using only the standard JDK—avoiding the need to bundle or explain an external JSON library for a course submission.
- Selective Exception Handling in `AlertEngine`:**
 - `AlertEngine.evaluate()` **throws**: Used at the moment a transaction is added. Here, a `CRITICAL` result is considered an exception worth interrupting the user flow for.
 - `checkAllBudgets()` **does not throw**: Used when rendering the full monthly summary dashboard, where every budget's status needs to be displayed in a single list (including `CRITICAL` ones). It computes the same thresholds but returns them as data objects rather than throwing exceptions.

- **Template-based recurring transactions:** Rather than running a background scheduler (which is overkill for a CLI tool that isn't running continuously), `RecurringTransaction` acts as a template. It stores a `nextDueDate` and exposes `isDue(asOf)` and `materialize()`. The user or `MainApp` explicitly triggers "process due transactions," matching the lifecycle of an on-demand CLI application.
- **Static per-class ID counters with `ensureNextIdAtLeast`:** In the absence of database auto-incrementing, `Transaction` and `RecurringTransaction` rely on static counters. To ensure IDs remain stable and avoid collisions across application restarts, the full constructor used during disk-loading bumps the counter past the highest ID encountered. Newly created transactions in a resumed session are thus guaranteed unique IDs.

9. Implementation Details

- **"Dumb but honest" Model Layer:** Model classes hold state, enforce domain invariants (e.g., `Budget` equality is strictly defined by `(category, yearMonth)` to enforce one budget per pair), and handle their own serialization via `toStorageRow()` and `fromStorageRow()`.
- **Generic `DataStorageUtil`:** Because serialization logic lives in the models, `DataStorageUtil` remains generic. It processes records line-by-line across three primary data files (`transactions.dat`, `budgets.dat`, and `recurring.dat`).
- **Centralized Alert Logic:** `AlertEngine` encapsulates threshold logic via `WARNING_THRESHOLD_PERCENT` (80%) and `CRITICAL_THRESHOLD_PERCENT` (100%). This ensures consistent evaluation across both real-time transaction entry and summary reports.
- **DRY Report Generation:** `ReportGenerator` delegates directly to `AlertEngine.checkAllBudgets()` to prevent logic drift between live alerts and report generation.
- **Decoupled Architecture:** `MainApp` contains zero business logic. It serves purely as an input/output controller that delegates tasks to the service and model layers. This separation keeps core logic testable and reusable independently of the CLI interface.

10. Testing Approach

Given the stateful, CLI-driven nature of the application, testing focused primarily on functional and manual verification via the menu interface and direct disk inspection of flat files:

- **Validation Boundaries:** Tested zero, negative, and extreme numeric values. Confirmed that `InvalidTransactionException` is thrown appropriately and system state/balance remains unaltered.

- **Budget Threshold Transitions:** Added incremental expenses within a budgeted category to verify state transitions from OK $\rightarrow$ WARNING (at 80%) $\rightarrow$ CRITICAL (at 100%, via exception handling).
- **Recurring Transaction Materialization:** Created recurring templates with past start dates spanning multiple elapsed intervals. Executed "*Process Due Recurring Transactions*" to verify that the correct number of transaction records were generated and nextDueDate advanced accurately across all frequencies (DAILY, WEEKLY, MONTHLY, YEARLY).
- **Persistence Round-Trip:** Populated transactions, budgets, and recurring items, then executed a save-and-exit sequence. Upon relaunching, verified that balances, lists, and configuration parameters loaded identically from disk.
- **Corrupted-Record Resilience:** Manually corrupted a line inside transactions.dat. Confirmed that the file parser safely skipped the malformed record, logged a descriptive warning, and loaded the remaining valid data without crashing.
- **ID Stability Across Restarts:** Added new transactions in a fresh session following a reload. Verified that auto-increment IDs correctly picked up above the highest existing disk ID without collision.

(Note: Attach relevant test run screenshots in Section 11 below.)

11. Screenshots

```
src > com > financetracker > main > MainApp.java
package src.com.financetracker.main;

import src.com.financetracker.exception.BudgetExceededException;
import src.com.financetracker.exception.InvalidTransactionException;
import src.com.financetracker.model.Account;
import src.com.financetracker.model.Budget;
import src.com.financetracker.model.Category;
import src.com.financetracker.model.RecurringTransaction;
import src.com.financetracker.model.Transaction;
import src.com.financetracker.service.AlertEngine;
import src.com.financetracker.service.ReportGenerator;
import src.com.financetracker.util.DataStorageUtil;

import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.Scanner;

/**
 * Command-line entry point for the Personal Finance & Expense Tracker.
 * Represents an interactive menu, where together the model, service, and util layers,
 * and persist data between runs via GitHub Repository API.
 */
public class MainApp {

    private static final DateTimeFormatter DATE_FMT = DateTimeFormatter.ISO_LOCAL_DATE;

    private final Scanner scanner = new Scanner(System.in);
    private final DataStorageUtil storage = new DataStorageUtil();
    private final AlertEngine alertEngine = new AlertEngine();
    private final ReportGenerator reportGenerator = new ReportGenerator();

    private Account account;
    private final List<Budget> budgets = new ArrayList<>();
    private final List<RecurringTransaction> recurringTransactions = new ArrayList<>();

    public static void main(String[] args) {
        MainApp app = new MainApp();
        app.run();
    }

    private void run() {
        System.out.println("PERSONAL FINANCE & EXPENSE TRACKER - Budget Alert Edition");
        System.out.println("Loading Data...");

        boolean running = true;
        while (running) {
            printMenu();
            String choice = promptLine("Choose an option: [1-7]");
            try {
                switch (choice) {
                    case "1" -> addTransaction();
                    case "2" -> editTransaction();
                    case "3" -> addTransactionCategory();
                    case "4" -> editBudgetBudget();
                    case "5" -> editBudgetBudget();
                    case "6" -> addRecurringTransaction();
                    case "7" -> deleteRecurringTransaction();
                    case "8" -> processRecurringTransactions();
                    case "9" -> generateSummaryReport();
                    case "0" -> exit();
                    default -> {
                        System.out.println("Invalid option. Please choose a number from the menu.");
                        running = false;
                    }
                }
            } catch (Exception e) {
                System.out.println("Invalid option. Please choose a number from the menu.");
                running = false;
            }
        }
    }

    private void printMenu() {
        System.out.println("1. Add Transaction");
        System.out.println("2. Edit Transaction");
        System.out.println("3. Add Transaction Category");
        System.out.println("4. Edit Budget Budget");
        System.out.println("5. Edit Budget Budget");
        System.out.println("6. Add Recurring Transaction");
        System.out.println("7. Delete Recurring Transaction");
        System.out.println("8. Process Recurring Transactions");
        System.out.println("9. Generate Summary Report");
        System.out.println("0. Exit");
    }
}
```

```
src > com > financetracker > model > RecurringTransaction.java
package src.com.financetracker.model;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.Objects;

/**
 * Represents a template for a transaction that repeats on a fixed schedule
 * (e.g., monthly rent, weekly allowance, yearly insurance premium).
 * The RecurringTransaction itself never lives on its own, directly;
 * instead, when obj is "materialized" into a real (Link Transaction)
 * by the code driving it (see MainApp's recurring-processing routine).
 */
public class RecurringTransaction {

    private static int idCounter = 1;

    public enum Frequency {
        DAILY,
        WEEKLY,
        MONTHLY,
        YEARLY
    }

    private static final DateTimeFormatter DATE_FMT = DateTimeFormatter.ISO_LOCAL_DATE;

    private final int id;
    private final Transaction.Type type;
    private final double amount;
    private final Category category;
    private final String description;
    private final Frequency frequency;
    private final LocalDate localdate;
    private final LocalDate nextDueDate;
    private boolean active;

    public RecurringTransaction(Transaction.Type type, double amount, Category category,
                               String description, Frequency frequency, LocalDate startdate) {
        this.idCounter++, type, amount, category, description, frequency, startdate, active: true;
    }

    public RecurringTransaction(int id, Transaction.Type type, double amount, Category category,
                               String description, Frequency frequency, LocalDate nextDueDate, boolean active) {
        this.id = id;
        this.type = Objects.requireNonNull(type);
        this.amount = amount;
        this.category = Objects.requireNonNull(category);
        this.description = description == null ? "" : description;
        this.frequency = Objects.requireNonNull(frequency);
        this.nextDueDate = Objects.requireNonNull(nextDueDate);
        this.active = active;
        if (id == idCounter) {
            idCounter = id + 1;
        }
    }

    public int getId() {
        return id;
    }

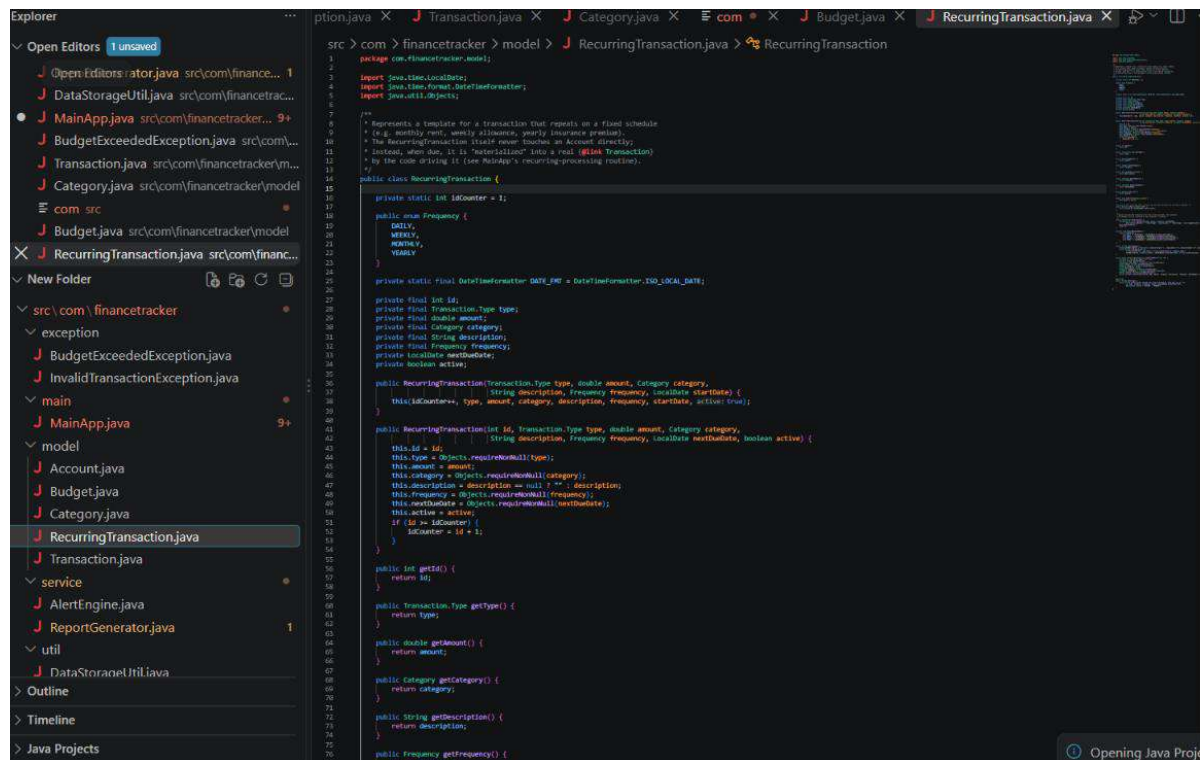
    public Transaction.Type getType() {
        return type;
    }

    public double getAmount() {
        return amount;
    }

    public Category getCategory() {
        return category;
    }

    public String getDescription() {
        return description;
    }

    public Frequency getFrequency() {
        return frequency;
    }
}
```



12. Challenges Faced

- Per-Line Error Handling in Data Storage:** A major challenge was deciding how to react when reading a corrupted or partially edited line in a .dat file during launch. Halting the entire application load would prevent access to valid saved transactions over a single bad line. Implementing per-line validation inside DataStorageUtil allows the parser to log or skip corrupted lines while successfully loading valid records, maintaining high app availability.
- Auto-Incrementing Transaction ID Stability:** Without a database sequence generator to handle auto-incrementing primary keys across sessions, generating unique transaction IDs posed a risk of ID collisions upon app restarts. Implementing Transaction.ensureNextIdAtLeast() solves this by scanning all loaded transaction IDs during initial startup and setting the static global counter to $\max(\text{existing_ids}) + 1$.

- **Centralizing Budget Threshold Logic:** To prevent discrepancies between live interactive alert popups and generated monthly reports, the 80% and 100% threshold calculations were centralized within domain service utility classes (`AlertEngine`) rather than scattered across CLI presentation handlers. Both runtime transaction prompts and report generators reference this single source of truth.
- **CLI Input & Scanner Parsing Edge Cases:** Interacting with Java's `Scanner` introduced edge cases around dangling newlines (`nextLine()`), empty inputs, and case-sensitivity typos. Dedicated UI helper methods like `promptCategory()` and `promptDateOrToday()` were built to wrap input loops, auto-trim strings, validate date formats against regex/parsing standards, and default empty date prompts seamlessly to the current date.
- **Checked vs. Unchecked Exception Boundaries:** Balancing when to throw checked exceptions versus runtime exceptions took multiple iterations. Custom checked exceptions were used for recoverable user flow failures (e.g., domain rule validation errors), forcing the caller to catch and re-prompt the user, whereas unchecked exceptions were reserved for unrecoverable structural failures.

13. Learnings & Key Takeaways

📌 **Domain-Specific Custom Exceptions:** Creating domain-tailored exceptions such as `InvalidTransactionException` and `BudgetExceededException` provided clear separation between generic Java system errors and business logic violations, resulting in cleaner control flow and precise user error messages.

📌 **File I/O Realities:** Serializing objects to flat files is straightforward, but reading them back robustly requires strict schema boundary checks, date format verification, and defensive parsing to handle manual file edits or missing fields gracefully.

📌 **Clean Architectural Separation:** Organizing the application into model, service, util, and exception packages saved significant debugging time. Keeping core business math inside model and service classes ensured CLI interface changes never broke budget calculations or data persistence logic.

📌 **Domain Model Behavior (materialize / isDue):** Encapsulating execution rules within `RecurringTransaction.materialize()` and `isDue()` highlighted the importance of rich domain models over passive data holders. Moving period checks and generation logic into the model simplified calling loops across the codebase.

14. Future Enhancements

- **Database-Backed Storage:** Replace `DataStorageUtil` flat-file storage with SQLite via JDBC. This will simplify multi-criteria querying (e.g., filtering expenses over ₹500 in Q1) and eliminate custom file-parsing routines.
- **Graphical User Interface (GUI):** Leverage the decoupled architecture to sit a JavaFX or Web UI on top of `Account`, `AlertEngine`, and `ReportGenerator` without modifying core domain logic.
- **Multi-Account Support:** Extend `Account` to manage multiple named accounts (e.g., "Personal", "Business") with isolated budgets and authentication.
- **Configurable Alert Thresholds:** Externalize the 80% and 100% warning thresholds from constants in `AlertEngine` into user-defined settings per budget category.
- **Data Visualization & Export:** Add ASCII bar charts in the CLI summary reports and enable CSV exports for easy spreadsheet import.
- **Automated Unit Testing Suite:** Implement JUnit tests for core calculation paths, alert triggers, and file round-trip serialization to replace manual CLI verification.

15. References

- Oracle Java Documentation (`java.util.Scanner`, `java.io`, `java.time.LocalDate`)
- Effective Java by Joshua Bloch (Exception Handling & Clean Architecture Patterns)